

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..42

SECURITY AND TLS

 Copy as Markdown

Security and TLS

Traffic between your app and your server should be encrypted, which on the web means HTTPS. HTTPS is HTTP wrapped in TLS, and somewhere in the chain a piece of software has to hold your certificate and do the encrypting. That job is called terminating TLS.

Most of the time it is not your server doing it. On Serverpod Cloud, TLS is handled for you and there is nothing to configure. On your own infrastructure it is normally handled in front of the server, by a load balancer or reverse proxy such as Nginx, a cloud load balancer, or Cloudflare, which forwards plain HTTP to Serverpod on an internal network.

Serverpod can also terminate TLS itself, which is useful when there is nothing in front of it to do the job.

Serve HTTPS from the server

Pass a `SecurityContextConfig` when you create the server, with a certificate chain and private key for each server you want to secure:

```
final securityContext = SecurityContext()
  ..useCertificateChain('path/to/server_cert.pem')
  ..usePrivateKey('path/to/server_key.pem', password: 'password');

Serverpod(
  args,
  securityContextConfig: SecurityContextConfig(
    apiServer: securityContext,
    webServer: securityContext,
    insightsServer: securityContext,
  ),
);
```

A Serverpod instance runs [three servers](#), and each takes its own context, so you can secure them independently.



Ask AI

Trust the server's certificate from your app

Your app only needs configuring when it cannot verify your certificate on its own. Certificates from a public authority, including the ones Serverpod Cloud provisions, are trusted automatically and need nothing here.

Self-signed certificates and private certificate authorities are the exception. There, name the certificate you want trusted by passing a `SecurityContext` to the generated `Client`:

```
final securityContext = SecurityContext()
  ..setTrustedCertificates('path/to/server_cert.pem');

final client = Client(
  'https://yourserver.com',
  securityContext: securityContext,
);
```

With an HTTP client override

The `securityContext` and `httpClientOverride` parameters cannot both be set on the same `Client`, since the override replaces the HTTP client the security context would have configured. Supply the certificates through the client you pass in instead.

On `dart:io` platforms, build an `HttpClient` with your trusted certificates and wrap it:

```
import 'dart:io';

import 'package:http/io_client.dart';

final securityContext = SecurityContext()
  ..setTrustedCertificates('path/to/server_cert.pem');

final client = Client(
  'https://yourserver.com',
  httpClientOverride: IOClient(
    HttpClient(context: securityContext),
  ),
);
```

Related

- [Configure HTTP calls](#): certificates and HTTP client overrides on the app side.
- [Configuration](#): request size limits and header settings.
- [Custom hosting](#): where a proxy fits when you host it yourself.